Unit 11 – Policy Iteration, Simulation-Based Policy Evaluation and SARSA

4SC000 Optimal Control and Reinforcement Learning

Duarte Antunes

Slides · Exercises · Jupyter notebooks

Where this unit sits in the course

Unit 10 set up the infinite-horizon problems in which reinforcement learning is easiest to state, and then gave one algorithm for them: value iteration, and its model-free shadow, Q-learning. Value iteration sweeps the Bellman equation until the values stop moving, and Q-learning is the same sweep with the expectation over successor states replaced by the one successor that happened to occur.

This unit gives the **other** classical algorithm for the same problems, and then follows it down the same road into the model-free world. The algorithm is **policy iteration**, and the idea behind it is quite unlike dynamic programming. Instead of improving a *value* until it is optimal and reading off a policy at the end, policy iteration improves a *policy* directly, by alternating two steps: work out what the current policy is worth, then replace it by the policy that is greedy with respect to that worth. Both steps are exact and neither involves a limit; the loop stops because there are only finitely many policies to run through.

What makes this worth a unit of its own is what happens when the model is removed. The evaluation step is the only one that needs the transition probabilities, and it is the only one that is an *expectation* – so it is exactly the step that simulation can replace. Doing so gives simulation-based policy iteration, and the two standard ways of estimating a policy's cost from simulated trajectories, **Monte-Carlo** and **temporal difference (TD)**, are the two basic tools of the whole field. Relaxing both steps – evaluating only approximately, improving only partially – gives **generalized policy iteration**, the pattern behind most algorithms used in practice, of which **SARSA** is the simplest instance worth naming. Finally, estimating the value of one policy from data generated by another – **off-policy learning** – is handled by importance sampling, which closes the unit.

Three threads from earlier units run through all of this. Policy improvement *is* the rollout step of Unit 8, with the current policy as the base policy. The move from cost functions to Q-factors *is* the move that made Q-learning possible in Unit 10, and it does the same work here. And the inventory control problem of Units 1 and 2, solved by stochastic dynamic programming there and by Q-learning in Unit 10, is solved a third and a fourth way below – exactly, and then from simulation – which is what makes it possible to check the learned answers against the truth.

Policy iteration

Policy iteration is an alternative procedure to dynamic programming for computing an optimal policy. It is used mostly when the state and input spaces are discrete, and it works for problems with disturbances, so it is an alternative to stochastic dynamic programming as well. Its two steps are **policy evaluation** and **policy improvement**, and almost everything in the rest of this unit – and much of reinforcement learning, including the actor-critic methods of Units 13 and 14 – is a variation on that pair.

Policy evaluation

Consider a shortest path problem with a terminal state,

$$x_{k+1} = w_k, \qquad \text{Prob}[w_k = j \mid x_k = i,\ u_k = \ell] = p_{ij}(\ell), \qquad k \in \{0, 1, 2, \dots\},$$

and let $J_\mu(i)$ be the cost incurred by a fixed policy μ started at $x_0 = i$. Then the n numbers $J_\mu(1), \dots, J_\mu(n)$ can be computed by solving the system of n **linear** equations

$$\begin{aligned} J_\mu(1) &= g(1, \mu(1)) + \sum_{j=1}^{n} p_{1j}(\mu(1)) J_\mu(j) \\ J_\mu(2) &= g(2, \mu(2)) + \sum_{j=1}^{n} p_{2j}(\mu(2)) J_\mu(j) \\ &\vdots \\ J_\mu(n) &= g(n, \mu(n)) + \sum_{j=1}^{n} p_{nj}(\mu(n)) J_\mu(j). \end{aligned}$$

The word to dwell on is *linear*. These equations look exactly like the Bellman equation of Unit 10 with one thing deleted: the minimisation. Because the policy is fixed, there is no choice to make at any state – the action at state i is $\mu(i)$, full stop – and what remains is a square linear system in the unknowns $J_\mu(1), \dots, J_\mu(n)$. In matrix form, with P_μ the matrix whose (i, j) entry is $p_{ij}(\mu(i))$ and g_μ the vector of stage costs,

$$J_\mu = g_\mu + P_\mu J_\mu \qquad \Longleftrightarrow \qquad (I - P_\mu) J_\mu = g_\mu,$$

a single solve rather than an iteration. That is the whole reason policy iteration exists: the *minimisation* is what makes the Bellman equation nonlinear and forces value iteration to iterate, and evaluating a fixed policy removes it.

The discounted case is identical with a discount factor inserted. For

$$x_{k+1} = w_k, \qquad \text{Prob}[w_k = j \mid x_k = i,\ u_k = \ell] = p_{ij}(\ell),$$

and the cost $\sum_{k=0}^{\infty} \mathbb{E}[\alpha^k g(x_k, u_k)]$, the values $J_\mu(1), \dots, J_\mu(n)$ solve

$$\begin{aligned} J_\mu(1) &= g(1, \mu(1)) + \alpha \sum_{j=1}^{n} p_{1j}(\mu(1)) J_\mu(j) \\ &\vdots \\ J_\mu(n) &= g(n, \mu(n)) + \alpha \sum_{j=1}^{n} p_{nj}(\mu(n)) J_\mu(j), \end{aligned}$$

that is $(I - \alpha P_\mu) J_\mu = g_\mu$. Here the solve is even better behaved: αP_μ has spectral radius at most $\alpha < 1$, so $I - \alpha P_\mu$ is invertible for *every* policy, with no reachability assumption needed.

Policy improvement

Given a policy μ and its costs $J_\mu(1), \dots, J_\mu(n)$, define a new policy by acting greedily with respect to them:

$$\bar{\mu}(i) = \arg\min_{u \in U(i)} \Big[g(i, u) + \sum_{j=1}^{n} p_{ij}(u) J_\mu(j) \Big].$$

Then $J_{\tilde{\mu}}(i) \leq J_{\mu}(i)$ for every i. This step is called **policy improvement**, and the inequality is the reason the whole scheme works.

It should look familiar. Taking one action by minimising the immediate cost plus the cost-to-go of a *base policy* thereafter is precisely the **rollout policy** of Unit 8, with μ as the base policy. The guarantee quoted there – a rollout policy is never worse than its base policy – is the same guarantee quoted here, and it is what makes the iteration monotone.

The loop, and why it terminates

Alternating the two steps gives policy iteration, shown in Fig. 1. One evaluates the current policy, improves it, evaluates the improved policy, and so on, stopping when the improvement step returns the policy it was given – at which point there is no point continuing.

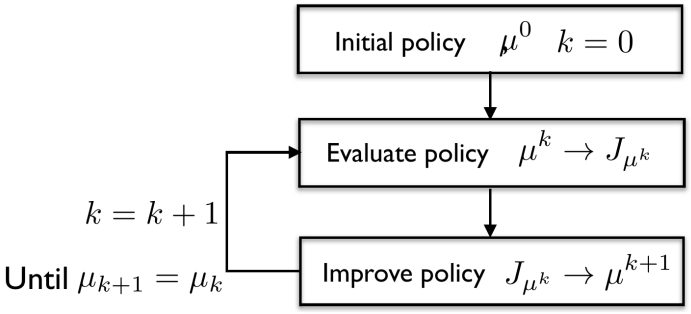


Fig. 1: The policy iteration loop. Starting from any policy μ^0, the two steps alternate: evaluation turns a policy into its cost-to-go, improvement turns a cost-to-go into a new policy. The same diagram serves the shortest path and the discounted problem; only the formulas below it differ, by a factor α.

In full, for the shortest path problem,

$$\textbf{Evaluate} \quad J_{\mu^k}(i) = g(i, \mu^k(i)) + \sum_{j=1}^{n} p_{ij}(\mu^k(i)) J_{\mu^k}(j),$$

$$\textbf{Improve} \quad \mu^{k+1}(i) = \arg\min_{u \in U(i)} g(i,u) + \sum_{j=1}^{n} p_{ij}(u) J_{\mu^k}(j),$$

and for the discounted problem,

$$\textbf{Evaluate} \quad J_{\mu^k}(i) = g(i, \mu^k(i)) + \alpha \sum_{j=1}^{n} p_{ij}(\mu^k(i)) J_{\mu^k}(j),$$

$$\textbf{Improve} \quad \mu^{k+1}(i) = \arg\min_{u \in U(i)} g(i,u) + \alpha \sum_{j=1}^{n} p_{ij}(u) J_{\mu^k}(j).$$

The procedure **must stop**, and the argument is a counting argument rather than an analytic one. There are only finitely many policies: if there are m possible decisions at each of n states, there are m^n of them. Each improvement step produces a policy that is no worse than the previous one, and a policy that is strictly better has never been visited before, so no policy can appear twice unless the iteration has stopped. Hence after at most m^n rounds the improvement step must return its own input.

This is a genuinely different kind of guarantee from the one in Unit 10. Value iteration converges *asymptotically* to the solution of the Bellman equation, so one needs a tolerance ϵ and a stopping test. Policy iteration

terminates *exactly*, in finitely many steps, and when it stops it has a certificate: the policy it holds is its own greedy improvement, which is the Bellman optimality condition written out. The price is that each step is a linear solve rather than a sweep, which costs $O(n^3)$ against value iteration's $O(n^2 m)$ per sweep.

Quick exercise: is the bound m^n any use?

The termination argument gives a bound of m^n iterations. For the Pac-man maze of Unit 10, with $n = 77 \times 77 = 5929$ states and $m = 5$ actions, that is 5^{5929} – a number with more than four thousand digits. Is the argument therefore worthless in practice?

Solution

The argument is worth a great deal, and the bound is worth nothing. They are different things.
What the argument establishes is that policy iteration **terminates at all**, in finitely many steps, with an exactly optimal policy and a certificate that it is optimal. No limit, no tolerance, no stopping heuristic. That is a structural fact about the algorithm, and nothing about the size of m^n weakens it.
The bound itself is almost never attained. In practice policy iteration converges in a handful of iterations – typically a number that grows very slowly with n, and often fewer than ten. The worked example below takes five; Problem 11.1 takes two. The reason is that each improvement step changes the action at *every* state where a better one exists, not at one state at a time, so a single round can leap a long way across the space of policies.
The honest statement is therefore: the m^n bound proves termination and predicts nothing. For *predicting* the iteration count one relies on the empirical observation that it is small, and on the fact – provable for the discounted case – that policy iteration converges at least as fast as value iteration, since each of its value estimates dominates the corresponding value iterate.

An example: inventory control, a third time

The inventory control problem of Units 1 and 2 is the right place to watch policy iteration run, because the answer is already known from two other routes. A shop holds at most $N = 4$ items over $h = 4$ stages. At each stage it may order u_k items, paying 5 per item plus a fixed transaction cost of 0.5 for any non-zero order; it pays 0.2 per item held; it sells at 10 per item, up to the demand. The demand is $d = (2, 2, 1, 1)$, except that at stage 2 it is uncertain, $\text{Prob}[d_2 = 0] = \text{Prob}[d_2 = 1] = 0.5$. Unsold stock at the end is salvaged at $5 - 0.2$ per item, giving the terminal cost $g_4 = (0, -4.8, -9.6, -14.4, -19.2)$. Costs are negative because the shop makes money.

Stochastic dynamic programming – value iteration – gives the optimal policy and the optimal costs-to-go of Fig. 2.

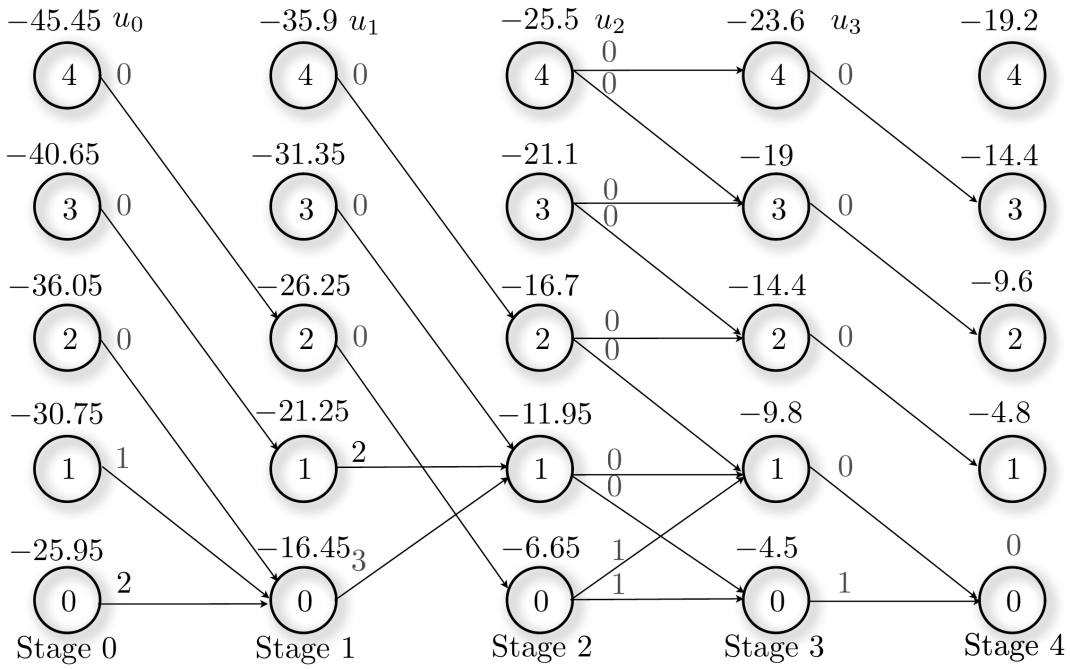


Fig. 2: The optimal policy and optimal costs-to-go for the inventory problem, from stochastic dynamic programming. Each circle is a stock level (0 at the bottom, 4 at the top) at a stage; the number above a circle is its optimal cost-to-go, the grey number beside it the optimal order quantity, and the arrows the resulting transitions. Stage 2 has two outgoing arrows per state because the demand there is random.

Policy iteration reaches the same lattice by a completely different route. Start from the policy "buy nothing", $\mu^0(x_k, k) = 0$ for every stock level and stage. Evaluating it is a backward sweep with no minimisation – the shop simply sells what it has and runs down – and gives Fig. 3.

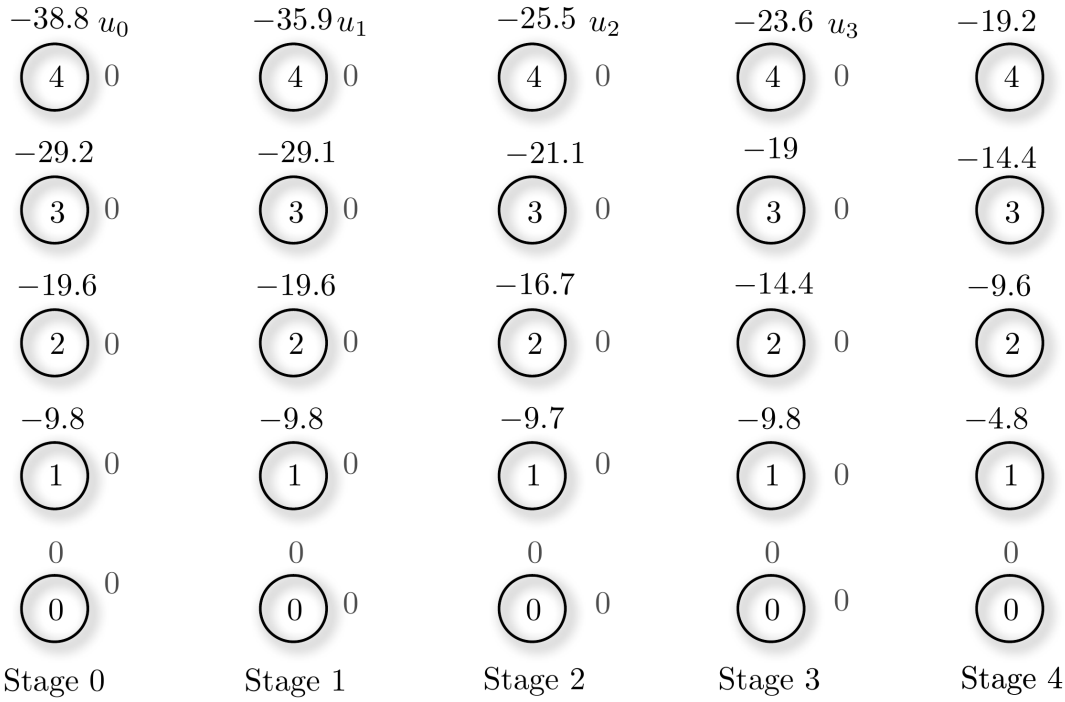


Fig. 3: Policy evaluation of the initial policy μ^0, "buy nothing". Every grey action is 0; the numbers above the circles are the costs J_{μ^0} of following that policy from there. An empty shop that never orders is worth exactly 0 at every stage, which is the bottom row.

Table 1: J_{μ^0} – the cost of never ordering. The last column is the terminal cost g_4.

stock	stage 0	stage 1	stage 2	stage 3	stage 4
4	−38.80	−35.90	−25.50	−23.60	−19.20
3	−29.20	−29.10	−21.10	−19.00	−14.40
2	−19.60	−19.60	−16.70	−14.40	−9.60
1	−9.80	−9.80	−9.70	−9.80	−4.80
0	0	0	0	0	0

Four more rounds finish the job. Fig. 4, Fig. 5, Fig. 6 and Fig. 7 show the improved policies $\mu^1, \ldots, \mu^4$ with their evaluations, and the fifth improvement step returns $\mu^5 = \mu^4$, at which point the algorithm stops, holding the optimal policy of Fig. 2.

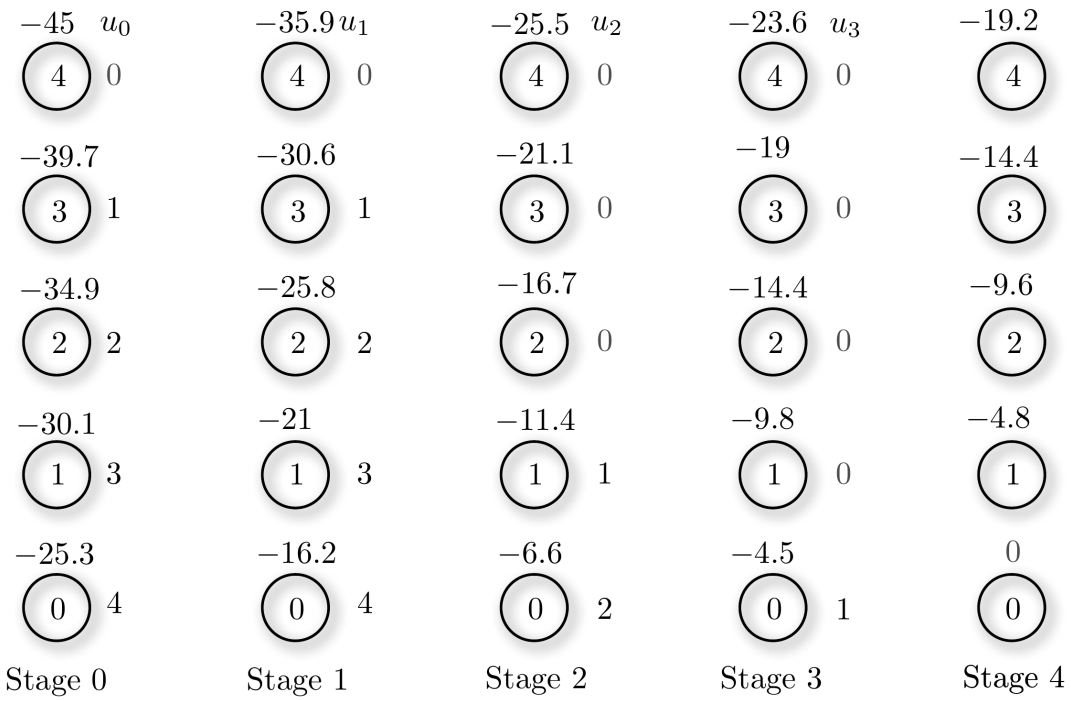


Fig. 4: Policy improvement applied to J_{μ^0} gives μ^1, and policy evaluation gives J_{μ^1}. The first improvement is dramatic: an empty shop at stage 0 goes from ordering nothing to ordering the full four items.

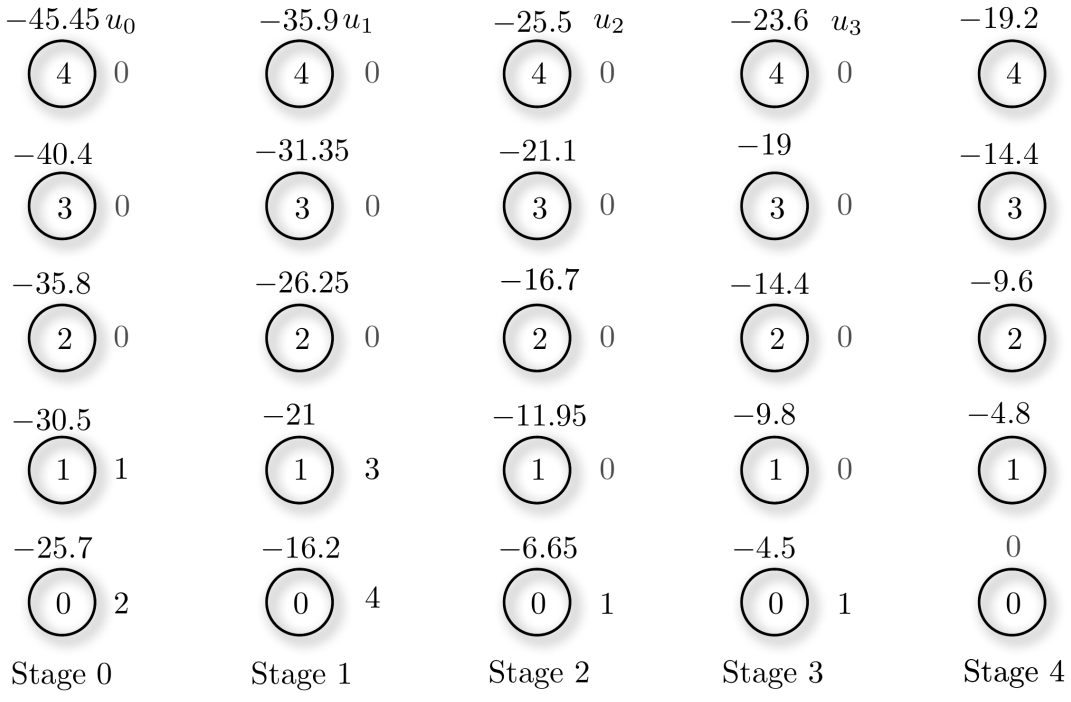


Fig. 5: μ^2 and J_{μ^2}.

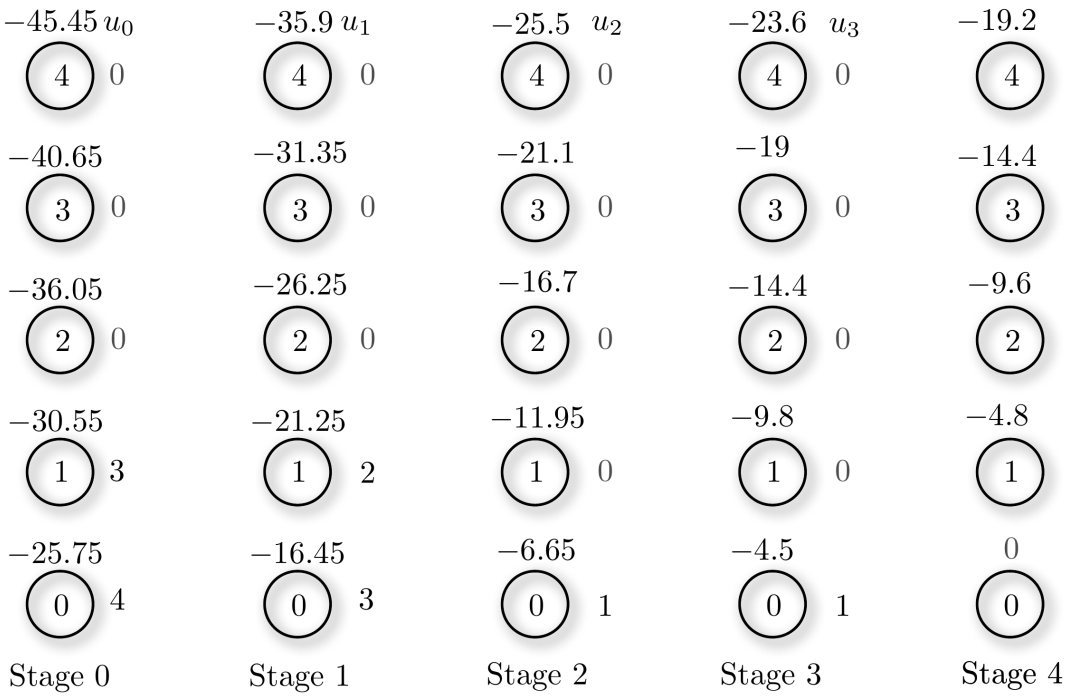


Fig. 6: μ^3 and J_{μ^3}.

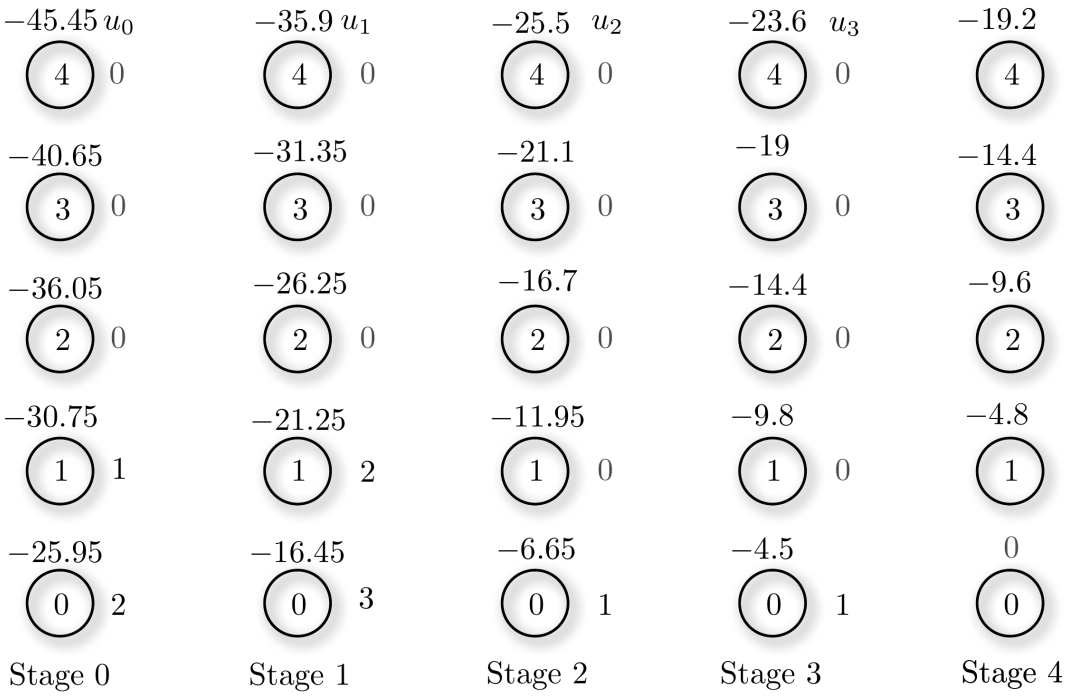


Fig. 7: μ^4 and J_{μ^4}. The next improvement step returns $\mu^5 = \mu^4$, so the algorithm stops and μ^4 is optimal.

The single number that summarises the run is the cost of starting with an empty shop at stage 0:

Table 2: One number per lattice: the cost of an empty shop at stage 0 under each successive policy.

iteration ℓ	0	1	2	3	4	5
$J_{\mu^\ell}(\text{stock } 0,\ \text{stage } 0)$	0	-25.30	-25.70	-25.75	-25.95	stop

Three things are worth reading off this sequence.

Almost all of the gain comes in the first step. The jump from 0 to -25.30 is the difference between a policy that never orders and one that orders; the remaining four rounds are worth 0.65 between them. This is typical, and it is the reason approximate versions of the algorithm are useful at all: a crude evaluation of a bad policy is enough to produce a much better one.

The values improve monotonically, but the policies do not march steadily towards the optimum. The order quantity at stock 0, stage 0 goes $0, 4, 2, 4, 2$ across the five policies, overshooting and returning, while the corresponding values fall monotonically. There is no contradiction: the guarantee is on $J_{\mu^{k+1}} \le J_{\mu^k}$, not on the policies converging in any geometric sense. The gaps here are genuinely small – under J_{μ^2} the order quantities 2 and 4 score -25.70 and -25.75 – so a small change in the evaluated values flips the minimiser. An approximate evaluation would flip it too, which is exactly the fragility that makes the simulation-based version of the algorithm harder to reason about than the exact one.

The answer agrees to the last digit with two other computations. The optimal stage-0 column, -25.95, -30.75, -36.05, -40.65, -45.45 for stock levels 0 to 4, is the same column produced by the stochastic dynamic programming of Unit 2 and by the exact reference inside Unit 10's Q-learning notebook. Three independent routes to five numbers is a stronger check than any one of them alone, and it is the reason this example is carried through the course. `A_11_StochInventoryControlPI.ipynb` runs the policy iteration shown here and prints every lattice.

Problems 11.3 and 11.9 in the exercise sheet run one round of this loop by hand, on an infinite-horizon and a finite-horizon version of the same shop.

Policy iteration with Q-factors

Everything above can be rewritten in terms of Q-factors, and for model-based policy iteration the rewriting changes nothing. For reinforcement learning the difference is subtle and crucial, and leads to very different implementations.

The Q-factor of a policy μ is the cost of taking action u once at state i and following μ thereafter. Evaluation and improvement become

$$\textbf{DC:}\quad Q_{\mu^k}(i,u) = g(i,u) + \alpha \sum_{j=1}^{n} p_{ij}(u)\, Q_{\mu^k}(j, \mu^k(j)),$$

$$\textbf{SP:}\quad Q_{\mu^k}(i,u) = g(i,u) + \sum_{j=1}^{n} p_{ij}(u)\, Q_{\mu^k}(j, \mu^k(j)),$$

$$\textbf{Improve:}\quad \mu^{k+1}(i) = \arg\min_{u \in U(i)} Q_{\mu^k}(i,u).$$

Note the bootstrap in the evaluation equation: the successor state j is evaluated at the action $\mu^k(j)$ that the *current policy* would take there. That is what makes it policy evaluation rather than a Bellman sweep, and it is the equation SARSA will later approximate from data.

The point of the rewriting is the improvement step. It is now a minimisation over a stored table, and **the model has disappeared from it entirely**. Compare the two versions side by side. With Q-factors,

$$\mu^{k+1}(i) = \arg\min_{u \in U(i)} Q_{\mu^k}(i,u),$$

and with cost functions,

$$\tilde{\mu}^{k+1}(i) = \arg\min_{u \in U(i)} g(i,u) + \alpha \underbrace{\sum_{j=1}^{n} p_{ij}(u) \tilde{J}_{\tilde{\mu}^k}(j)}_{\text{needs } p_{ij}\text{, or many more simulations}}.$$

In the second form, extracting a policy requires either knowledge of p_{ij} or enough extra simulations to approximate the expectation – and those simulations must be run *from state i under each candidate action u*, which is a much more demanding requirement than passively watching trajectories. In the first form, nothing is needed but the table.

This is exactly the observation that made Q-learning possible in Unit 10, doing the same work in a different algorithm. The price is also the same, and it is paid in **exploration**: a table indexed by (i,u) has m times as many entries to fill, and a deterministic policy visits only one action per state. That is the subject of a section below.

Quick exercise: where did the model go?

The Q-factor improvement step uses no model, while the cost-function one needs p_{ij}. But the Q-factors themselves were *defined* by an equation full of p_{ij}. Has the model dependence really been removed, or just moved somewhere less visible?

Solution

Moved – but moved somewhere that simulation can reach, which is the whole point.
The model dependence sits in the *evaluation* step, and the evaluation step is an **expectation of something that can be observed**: the cost of following μ from a given state-action pair. Run the policy from that pair and add up what happens, and you have an unbiased sample of exactly the quantity $Q_\mu(i,u)$ is defined to be. Averaging many such samples estimates it without ever writing down p_{ij}. That is Monte-Carlo evaluation, and it is the subject of the next section.
The cost-function improvement step is not of that kind. It needs $\sum_j p_{ij}(u) J_\mu(j)$ for *every candidate* u, including the ones the current policy never takes, so the data to estimate it does not exist in a trajectory generated by μ: it would have to be gathered by deliberately taking each u at each i and watching where it leads. Formally one could do that, and for a simulator one can – but it is a different and much stronger kind of access to the system than observing a stream of experience.
So the right statement is: Q-factors do not remove the model, they relocate the dependence from a step that cannot be sampled to a step that can. The same sentence describes the move from value iteration to Q-learning in Unit 10.

Simulation-based policy iteration

Replace the exact evaluation step by an estimate built from simulated trajectories and policy iteration becomes usable without a model. Fig. 8 puts the model-based and simulation-based versions side by side in the cost-function form, and Fig. 9 does the same in the Q-factor form.

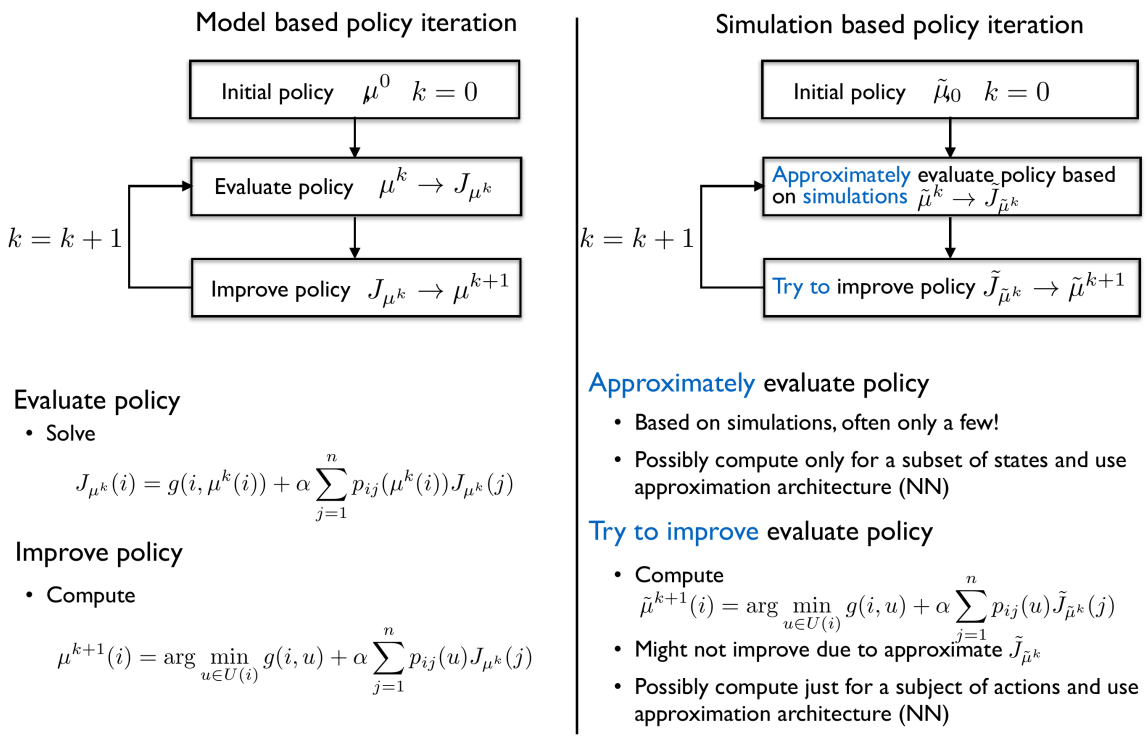


Fig. 8: Model-based policy iteration (left) against simulation-based policy iteration (right), in the cost-function form. The loop is unchanged; both boxes are weakened. Evaluation becomes approximate and is built from simulations, possibly only a few, possibly only for a subset of states and with an approximation architecture such as a neural network. Improvement becomes an *attempt* at improvement, which may fail precisely because the values it is greedy with respect to are approximate.

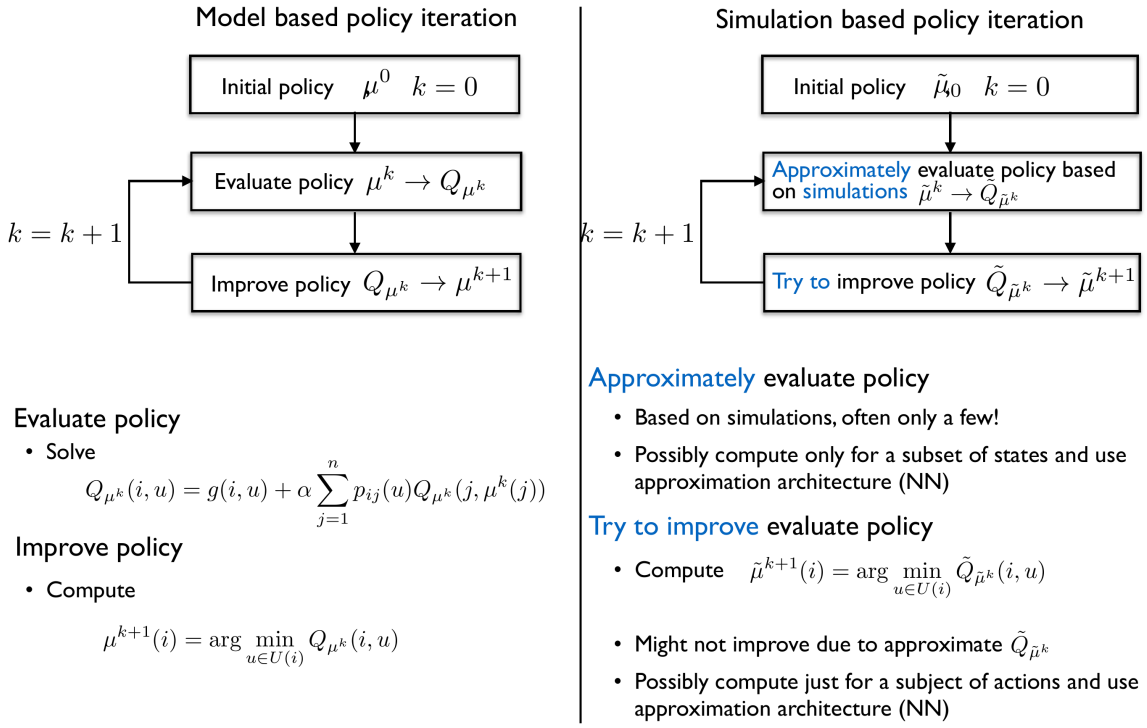


Fig. 9: The same comparison in the Q-factor form. The evaluation equation now bootstraps on $Q_{\mu^k}(j, \mu^k(j))$, the successor state evaluated at the action the current policy takes; the improvement step is a minimisation over the stored table and involves no model at all.

Two words on those diagrams deserve emphasis, because they are what separates this section from the exact algorithm above. "**Approximately** evaluate" means the values are estimates with sampling error in them, not solutions of a linear system. And "**try to** improve" means the improvement step may now *fail* to improve: the monotonicity guarantee $J_{\mu^{k+1}} \le J_{\mu^k}$ was proved for the exact values, and greedy action selection against noisy values carries no such guarantee. The worked example above already showed how little margin there can be – two order quantities separated by 0.05 – so this is not a theoretical worry.

Everything now turns on the evaluation step. There are two basic ways to estimate the cost of a policy from simulations: **Monte-Carlo** and **temporal difference**.

Monte-Carlo policy evaluation

Monte-Carlo evaluation is the direct approach: run the policy, see what it costs, average. Suppose we run M simulations of the system under a fixed policy μ,

$$x_0^\ell, x_1^\ell, \dots, x_N^\ell \qquad \ell \in \{1, \dots, M\},$$

where for shortest path problems x_N^ℓ is a terminal state, and otherwise the simulation is run for a long time N (the appendix discusses how to choose it).

Step 1. Every time $r \in \{1, 2, \dots\}$ that a simulation visits a state $x_k = i$, compute the **return** from that visit onwards,

$$R_r(i) = \sum_{\ell=k}^{N} g(x_\ell, u_\ell), \quad u_\ell = \mu(x_\ell) \qquad \Big(\text{for DC: } R_r(i) = \sum_{\ell=k}^{N} \alpha^{\ell-k} g(x_\ell, u_\ell)\Big).$$

Step 2. Estimate the cost-to-go of μ at state i by the average of those returns,

$$\hat{J}(i) = \frac{1}{\bar{n}(i)} \sum_{r=1}^{\bar{n}(i)} R_r(i), \qquad \bar{n}(i) = \#\text{ visits to state } i \text{ across all simulations.}$$

The Q-factors come from the same recipe with one change of bookkeeping: treat a state-action pair as the thing being visited.

Step 1. Every time r that a simulation is at $x_k = i$ and action $u_k = j$ is taken, compute

$$R_r(i,j) = \sum_{\ell=k}^{N} g(x_\ell, u_\ell), \qquad u_\ell = \begin{cases} j & \text{if } \ell = k \\ \mu(x_\ell) & \text{if } \ell > k \end{cases}$$

$$\Big(\text{for DC: } R_r(i,j) = \sum_{\ell=k}^{N} \alpha^{\ell-k} g(x_\ell, u_\ell)\Big).$$

Step 2. Average them,

$$\hat{Q}(i,j) = \frac{1}{\bar{n}(i,j)} \sum_{r=1}^{\bar{n}(i,j)} R_r(i,j), \qquad \bar{n}(i,j) = \#\text{ visits to the pair } (i,j).$$

Note what the definition of $R_r(i,j)$ says: the first action is j, whatever the policy would have done, and every action after it is the policy's. That is the definition of a Q-factor, read as an instruction for a simulator.

Temporal difference evaluation

To motivate the temporal difference method, first rewrite Monte-Carlo so that it computes its average **incrementally**, as Q-learning did in Unit 10:

$$\hat{J}_{t+1}(i) = \hat{J}_t(i) + \gamma(R_r(i) - \hat{J}_t(i)) \qquad \text{when } x_t = i \text{ is visited,}$$

with a step size γ that may vary with t but is taken constant here. (With $\gamma = 1/r$ at the r-th visit this is exactly the running mean; with γ constant it is an exponentially weighted one.)

Now decompose the return into its first term and the rest:

$$R_r(i) = \sum_{\ell=k}^{N} g(x_\ell, u_\ell) = g(x_k, \mu(x_k)) + \underbrace{\sum_{\ell=k+1}^{N} g(x_\ell, u_\ell)}_{\approx\ \hat{J}_t(x_{k+1})}.$$

The underbraced sum is the return from the *next* state onwards – and we already hold an estimate of it, namely $\hat{J}_t(x_{k+1})$. The TD method uses that estimate instead of waiting for the trajectory to finish:

$$\hat{J}_{t+1}(x_k) = \hat{J}_t(x_k) + \gamma\Big(\overbrace{g(x_k, \mu(x_k)) + \hat{J}_t(x_{k+1}) - \hat{J}_t(x_k)}^{\text{temporal difference}}\Big).$$

Dropping the time index and writing the update with an arrow, as is customary:

$$\hat{J}(x_k) \leftarrow \hat{J}(x_k) + \gamma\Big(g(x_k, \mu(x_k)) + \hat{J}(x_{k+1}) - \hat{J}(x_k)\Big),$$

and with a discount, $g(x_k, \mu(x_k)) + \alpha\hat{J}(x_{k+1}) - \hat{J}(x_k)$ inside the bracket. The quantity in the bracket is the **temporal difference**, usually written δ: the discrepancy between what the stored estimate said this state was worth and what one step of actual experience plus the stored estimate of the next state say it is worth.

Fig. 10 shows the difference in one picture.

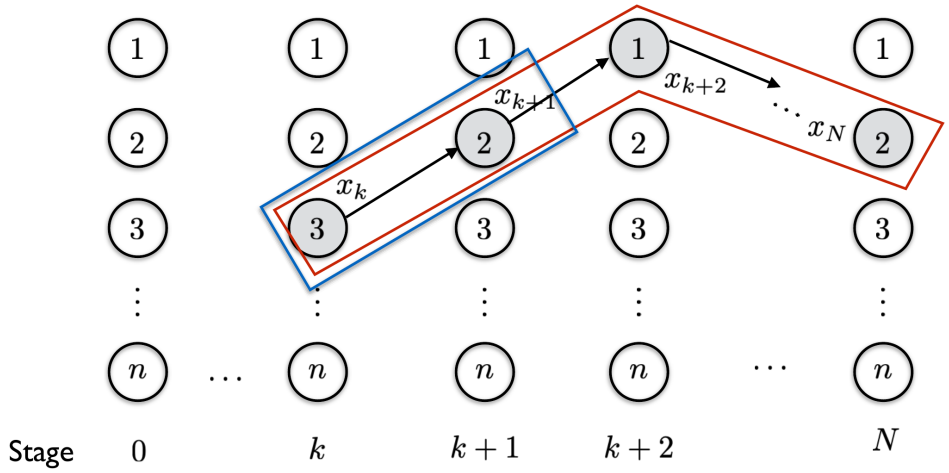


Fig. 10: Monte-Carlo and TD returns for the same visit to x_k. The red bracket is the Monte-Carlo return: the whole remaining trajectory, from x_k to x_N. The blue bracket is the TD return: the single transition $x_k \to x_{k+1}$, plus the stored estimate at x_{k+1}.

The **Monte-Carlo return** depends on the full trajectory after x_k, so one must wait until the trajectory is over before it can be computed at all. The **TD return** is local: it depends only on x_{k+1}, can be computed immediately, and reuses the cost-to-go estimates already built up.

Monte-Carlo or TD?

Three points settle the comparison for the purposes of this course.

TD is the more useful of the two. It is simpler, more practical and more widely used, and it is at the heart of most of the more advanced methods – including SARSA below, and Q-learning in Unit 10, which is the same idea applied to the Bellman equation rather than to a fixed policy's evaluation equation.

Monte-Carlo is unbiased; TD is not. For discounted problems, the Monte-Carlo return is an unbiased sample of the quantity being estimated – it *is* the cost of the policy along one realisation. The TD target $g + \alpha\hat{J}(x_{k+1})$ is not, because $\hat{J}(x_{k+1})$ is a current estimate rather than the true value, and an error there propagates straight into the update. For methods that need an unbiased estimate – for a convergence proof, or for well-definedness – Monte-Carlo is the one to use. Importance sampling, later in this unit, is such a case.

TD on a fixed batch converges to the certainty-equivalence solution. If TD is applied repeatedly to a fixed batch of data, it converges to the costs-to-go one would obtain by first estimating a model from that data – transition frequencies – and then solving that model *exactly*. That is a precise statement of what TD is doing with its bootstrapping: it is implicitly fitting a model and solving it, without ever forming the model. Batch Monte-Carlo, by contrast, converges to the estimate that minimises squared error on the observed returns, which is a different answer. Neither is uniformly better; TD's answer is usually the more useful one when the data really do come from a Markov chain, because it exploits that structure and Monte-Carlo does not.

The two are not in fact opposites but the two ends of one family. The Monte-Carlo error at a state is exactly the discounted sum of all the TD errors along the remaining trajectory,

$$R_k - \hat{J}(x_k) = \sum_{\ell=k}^{\infty} \alpha^{\ell-k} \delta(x_\ell), \qquad \delta(x_\ell) = g(x_\ell) + \alpha \hat{J}(x_{\ell+1}) - \hat{J}(x_\ell),$$

provided $\hat{J}$ is held fixed along the trajectory. Truncating that sum after n terms gives n-step TD; weighting the truncations geometrically gives TD(λ), with $\lambda = 0$ recovering TD and $\lambda = 1$ recovering Monte-Carlo. Problem 11.6 proves the identity; it is three lines of telescoping, and it is the cleanest way to see that the bias-variance trade-off between the two methods is a continuum rather than a choice.

Problem 11.5 runs both methods by hand on the same eleven-step trajectory, which is worth doing once: the two end up at visibly different numbers, and seeing why is more instructive than the general statements above.

Exploration

Dynamic programming obtains a policy for every state, because it sweeps through all of them. Simulation-based policy iteration does not: the states it learns about are the states its own iterates μ^k happen to visit. That is both the method's great economy and its central difficulty, and the inventory shop shows both faces of it.

Suppose the optimal policy, and the iterates leading to it, favour driving the inventory towards zero. Then states with large inventories may never be visited, even with demand disturbances. This is **good**: those states are irrelevant from the point of view of the optimal policy, and computing their costs-to-go accurately would be wasted effort. A method that only learns about the part of the state space a good policy actually occupies is doing something dynamic programming cannot.

Now suppose instead that the initial policy μ^0 favours high-inventory states, and that the cost of the low-inventory states happens to be estimated as high. Then the improvement step will keep favouring high-inventory states, and the simulations will keep visiting only those, so the bad estimates of the low-inventory states will never be corrected – and the shop will never discover that running lean is better. This is **poor exploration**, and it is a genuine failure mode, not a slow convergence.

The two situations are indistinguishable from inside the algorithm. Both look like "the simulations only visit these states"; only in the first case is that a feature. There are several remedies, none of them fully satisfactory.

Exploring starts and ϵ-soft policies

Exploring starts. The simplest option: make sure the initial state is set, several times, to each possible state. It has three drawbacks. It is only feasible if the state space is small enough to sweep, which defeats much of the purpose. It requires the simulator to be startable from an arbitrary state, which a real system generally is not. And it does not concentrate on the *meaningful* states – those actually generated by sensible control inputs – so a good deal of the computation is spent on states no reasonable policy ever occupies.

Random behaviour added to the policy. The prime example is an ϵ-soft policy. Given a deterministic policy μ^k, its ϵ-soft version $\mu^{\epsilon,k}$ picks the action $\mu^k(i)$ with probability $1 - \epsilon$ and, with probability ϵ, one of the other **actions**, all of the others being equally likely. The idea is that meaningful actions are still taken most of the time, but once in a while something else is tried, possibly leading to good and unexplored states.

Replacing μ^k by an ϵ-soft version is the standard move for policy evaluation. It has a cost that is easy to overlook: what then gets estimated is the cost of the ϵ-soft *version*, not of μ^k. One is evaluating a policy one does not intend to use. The usual mitigation is to start with a large ϵ, $0 < \epsilon < 1$, and reduce it gradually, so that the policy being evaluated converges to the policy being sought. SARSA, below, does exactly this, and off-policy learning, at the end of the unit, is the alternative: keep the two policies distinct and correct for the difference explicitly.

Why Q-factors make exploration harder

When working with cost functions J, policy evaluation can still achieve reasonable exploration on its own, provided the disturbances excite the system enough to make it wander through many – eventually all – of the states. The randomness in the problem does the exploring.

With Q-factors this is definitely not the case, and the reason is structural. Q-factors are indexed by (state, action) pairs, which is to say they live on an augmented state space of size nm rather than n. Under a *deterministic* policy μ^k, the only pairs ever visited are $\{(i, \mu^k(i))\}$ – exactly n of the nm. The remaining $n(m-1)$ entries receive no data at all, no matter how long the simulation runs and no matter how noisy the system is. Disturbances move the state; they do not move the action.

Two consequences follow. Trajectories cannot be reused across policies unless ϵ-soft policies are used, since a trajectory generated by μ^k carries no information about actions μ^k does not take. And even with an ϵ-soft policy, the pairs belonging to μ^k are visited far more often than the others – by a factor of roughly $(1-\epsilon)(m-1)/\epsilon$ – so the estimates of the alternatives, which are exactly the estimates the improvement step compares against, are the least reliable numbers in the table.

This is the price of the model-free improvement step, and it is worth stating plainly: moving the model dependence out of the improvement step bought a factor of m in the size of the thing that has to be estimated, and a qualitatively harder exploration problem.

Quick exercise: when does poor exploration not matter?

`A_11_StochInventoryControlSimBasedPI.ipynb` runs simulation-based policy iteration on the inventory problem and compares every entry against the exact answer. Most entries come out accurate to a few tenths, but a handful are still sitting at their initial value of zero, with errors above 30. The convergence theorem's first condition – every state-action pair visited infinitely often – is visibly violated. Why does the notebook nevertheless report that the learned policy is a good one?

Solution

Because of *which* entries are missing. They are the high-stock states at the middle stages: a stock of 3 or 4 at stage 1 or 2.
Reaching one of those would require placing a large order into an already well-stocked shop – an action no good policy takes, and the iterates μ^k do not take it. Exploration would have to supply the visit by chance, and the exploration rate decays after the first 25 000 simulations, so after that it does not.
Now ask what is lost. The policy is only ever *used* from states it can actually reach, and those states are unreachable under any policy the algorithm would want to follow. Their costs-to-go never enter a decision. So the violated condition costs nothing here: the theorem asks for every pair to be visited because it proves convergence of *every* entry of the table, and we only need the entries that get consulted.
The important caveat is that this reasoning is only available *after the fact*, and only because the exact answer is known. From inside the algorithm, "these states were never visited because no good policy goes there" and "these states were never visited because my initial policy was bad and nothing ever corrected it" look identical. That is precisely the ambiguity the section above describes, and it is why the demonstration is run on a problem small enough to check.
`A_11_RobotPlanningSimulationBasedPI.ipynb` shows the other side of the coin, and shows it with a number: a bug in the original script makes the exploratory action always "up" rather than uniform over the three available actions, and the maximum error against the exact cost-to-go goes from 1.89 to 6.39. Same algorithm, same step size, same number of simulations – the answer is wrong because one branch of the action space was never tried.

Generalized policy iteration

Generalized policy iteration (GPI) is the name for the whole family of methods inspired by policy iteration that do not necessarily carry out either step exactly. Policy evaluation may be performed only for a few states – for instance only those appearing in a simulation – and only approximately; policy improvement likewise, perhaps only for a subset of the actions, perhaps through an approximation architecture. Approximate simulation-based policy iteration with inaccurate evaluation is a practical scheme and a special case of GPI.

The idea that holds the family together is this. Even if each component is imperfect, evaluation tends to make the values more accurate, and improvement tends to make the policy better by being greedy with respect to the current values. The two pull in compatible directions, and the hope is that the process converges even when neither step is exact. Under certain conditions it can be established that it does.

It is worth naming what has been given up. Exact policy iteration had a finite termination argument and a certificate of optimality. GPI has neither: it is a heuristic with a convergence theory for special cases, and the reason to study it is that essentially every reinforcement learning algorithm used in practice is an instance of it. Q-learning, read in this light, is GPI with an evaluation step of exactly one backup; SARSA is the same with the bootstrap taken from the policy rather than from the minimum.

SARSA

SARSA is the on-policy TD control algorithm, and it is the simplest member of the GPI family worth naming. It applies the TD update to Q-factors:

$$\widehat{Q}(x_k, u_k) \leftarrow \widehat{Q}(x_k, u_k) + \gamma\Big(g(x_k, u_k) + \widehat{Q}(x_{k+1}, u_{k+1}) - \widehat{Q}(x_k, u_k)\Big).$$

The name is the list of what one update consumes:

$$\underbrace{\text{State}}_{x_k} - \underbrace{\text{Action}}_{u_k} - \underbrace{\text{Reward}}_{g} - \underbrace{\text{State}}_{x_{k+1}} - \underbrace{\text{Action}}_{u_{k+1}} \quad \text{(SARSA)}.$$

The control policy is ϵ-greedy with respect to the current table,

$$\text{Prob}[u_k = a] = \begin{cases} 1 - \epsilon & \text{if } a = \arg\min_u \widehat{Q}(x_k, u) \\ \dfrac{\epsilon}{\#\text{actions at } x_k - 1} & \text{otherwise,} \end{cases}$$

so with probability $1 - \epsilon$ the greedy action is taken and otherwise one of the others, uniformly. As in the exploration section, ϵ starts large and is progressively reduced.

Three observations make SARSA worth its name.

It is GPI. There is no separate evaluation phase and no separate improvement phase: every transition does a little of both. The update moves $\widehat{Q}$ towards the value of the policy currently being followed (evaluation), and the policy is read off $\widehat{Q}$ at every step (improvement). Policy iteration's two boxes have been interleaved to the point of being simultaneous.

It differs from Q-learning in exactly one symbol. Q-learning's target is $g + \min_v \widehat{Q}(x_{k+1}, v)$; SARSA's is $g + \widehat{Q}(x_{k+1}, u_{k+1})$, the action actually taken next. That is the whole difference between the two algorithms, and it is the difference between **off-policy** and **on-policy**: Q-learning estimates the value of the greedy policy no matter what policy generated the data, while SARSA estimates the value of the policy generating the data – which, because the ϵ-greedy policy is derived from the same table, is itself.

The difference shows up in the data requirements. A Q-learning update needs (x_k, u_k, g, x_{k+1}); a SARSA update needs $(x_k, u_k, g, x_{k+1}, u_{k+1})$. So for Q-learning a transition is a self-contained training example and a data set is a *set*, while for SARSA the examples overlap and a data set is a *sequence* that cannot be

cut without loss. Problem 11.4 makes this concrete in an unusually direct way: it asks for SARSA to be run on the same ten-step data table Unit 10 used for Q-learning, and the table turns out to be one column short.

Example: frozen lake

OpenAI Gymnasium is one of the most prominent collections of standard environments for testing reinforcement learning ideas, and the **frozen lake** is one of its simplest.

The game starts with the player at position $[0, 0]$ of a 4×4 grid world, with the goal at $[3, 3]$. The player chooses up, left, right or down at each step until it reaches the goal or falls into a hole. The lake is **slippery**, so the player may instead move perpendicular to the intended direction. A reward of one is given for reaching the goal, and zero for falling into a hole or for letting 100 iterations elapse.

Fig. 11 shows SARSA and Q-learning applied to it.

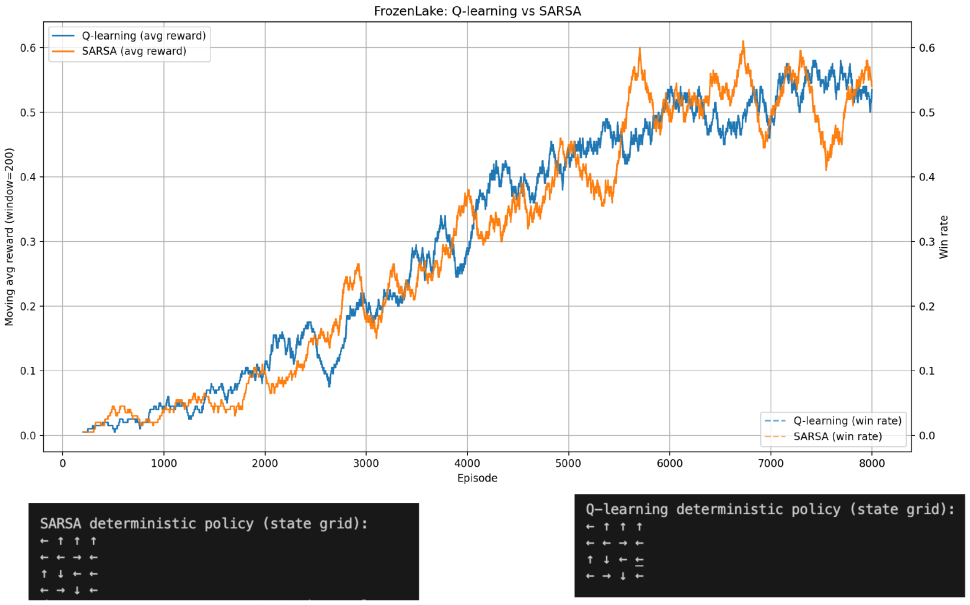


Fig. 11: SARSA and Q-learning on the slippery 4×4 frozen lake. The two algorithms reach similar performance and settle on similar policies, which is what one expects when the behaviour policy is ϵ-greedy with a decaying ϵ: the two targets agree whenever the greedy action is the one taken.

`A_11_FrozenLakeQLearningSARSA.ipynb` runs both and reproduces this plot. Two details from it are worth carrying away. Both algorithms reach a win rate near 0.5 – the lake is slippery enough that half the games are lost by a policy that plays perfectly. And several arrows in the learned policies point straight **into a wall**, which is correct rather than a bug: commanding into a wall makes the intended move a no-op and leaves only the two perpendicular slips, so it is how one moves sideways with no chance of sliding into a hole. It is a good reminder that a learned policy exploiting the dynamics will not always look sensible when read as intent.

One warning about the environment. Gymnasium hands out **rewards to be maximised**, while this course minimises a **cost**; the two conventions differ by a sign, and the greedy action in the notebook is accordingly an arg max where every formula in these notes has an arg min.

Off-policy learning

Reinforcement learning faces a dilemma that has no counterpart in Part I. We want to learn the optimal policy, and to gather informative data we must run *some* policy – but if we could already run the optimal policy, we would not need to find it. Data gathered under a good policy is data about a narrow slice of the state-action space; data gathered under an exploratory policy is about a policy we do not want.

Off-policy learning is the name for the resolution: learn the value of one policy – the **target policy** π – while following a different policy – the **behaviour policy** μ – to collect the data. Typically π is the optimal policy, or the current greedy policy, and μ is something that explores.

SARSA is *on-policy* and sidesteps the dilemma rather than solving it: it learns the value of the ϵ-greedy policy, which is both target and behaviour policy, and lets ϵ go to zero so that the policy it is learning about drifts towards the one we want. Q-learning, by contrast, is genuinely off-policy already – its min makes its target the greedy policy regardless of what generated the data – which is why it needed no machinery in Unit 10. For Monte-Carlo and TD *evaluation*, where the target is a specific policy's value rather than a minimum, the correction has to be made explicitly. Most methods do it with **importance sampling**.

Importance sampling

Start with a problem that has no control in it at all. Let x be a random variable distributed according to a **behaviour** distribution $b = (b_1, \dots, b_n)$, which we can sample, obtaining data $x_1, \dots, x_K$. We want the expected value with respect to a different, **target** distribution $t = (t_1, \dots, t_n)$,

$$J = \sum_{i=1}^{n} t_i c_i,$$

where $c_1, \dots, c_n$ are the values x can take. The difficulty is that the samples come from b, not from t. Averaging them estimates $\sum_i b_i c_i$, which is the wrong number.

Importance sampling repairs this by reweighting each sample:

$$\bar{J} = \frac{1}{K} \sum_{k=1}^{K} \rho_k x_k, \qquad \rho_k = \frac{t_i}{b_i} \text{ if } x_k = c_i, \qquad b_i > 0,$$

where ρ_k is the **importance ratio**. The estimate is unbiased, and the computation showing it is one line:

$$\mathbb{E}[\bar{J}] = \frac{1}{K} \sum_{k=1}^{K} \mathbb{E}[\rho_k x_k] = \frac{1}{K} \sum_{k=1}^{K} \Big(\sum_{i=1}^{n} \frac{t_i}{b_i} c_i \, b_i \Big) = \sum_{i=1}^{n} t_i c_i = J.$$

The b_i in the expectation – the probability of drawing c_i – cancels the b_i in the denominator of the ratio. That cancellation is the whole mechanism, and it is also where the method's weakness lives: a value that b makes rare but t makes common carries a large ratio, so the estimate rests on a few heavily weighted samples. Problem 11.7 is a five-point distribution engineered to show this, where b assigns 0.1 to a value carrying 0.7 of t's mass.

The condition $b_i > 0$ is not a technicality. If the behaviour distribution never produces a value the target distribution cares about, no amount of reweighting recovers it – there is nothing to reweight. Transferred to policies, the requirement is that the behaviour policy must give every trajectory the target policy can produce a positive probability of occurring, and an ϵ-soft policy is the standard way to guarantee it: given any deterministic policy, it takes that policy's action with probability $1 - \epsilon$ and any other action with probability ϵ spread equally, so no action anywhere has probability zero.

Importance sampling for returns

Now transfer this to policy evaluation. We wish to estimate the Monte-Carlo return of a target policy π from data generated by a behaviour policy μ. Fig. 12 shows the object being averaged over.

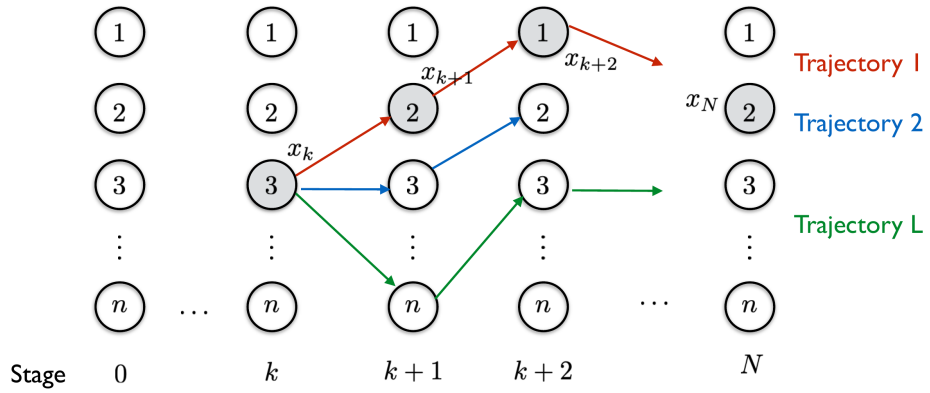


Fig. 12: The trajectories available from a state x_k. There are finitely many – L of them, with costs $c_1, \dots, c_L$ – and each has a probability under the behaviour policy and another under the target policy. Importance sampling replaces the simple average of observed returns by an average weighted by the ratio of those two probabilities.

Writing $b = (b_1, \dots, b_L)$ and $t = (t_1, \dots, t_L)$ for the probability of each trajectory under μ and under π, the Monte-Carlo estimate becomes a weighted average,

$$\hat{J}(i) = \frac{1}{\bar{n}(i)} \sum_{r=1}^{\bar{n}(i)} \rho_r R_r(i), \qquad \rho_r = \frac{t_j}{b_j} \;\text{ if } R_r(i) = c_j.$$

This looks like it needs the model, since the probability of a trajectory certainly does. It does not, and seeing why is the key step. Suppose both policies are stochastic,

$$\textbf{Behaviour} \quad \text{Prob}[u_t = j \mid x_t, \mu] = \mu_j(x_t), \qquad \mu(x_t) = (\mu_1(x_t), \dots, \mu_{m(x_t)}(x_t)),$$

$$\textbf{Target} \quad \text{Prob}[u_t = j \mid x_t, \pi] = \pi_j(x_t), \qquad \pi(x_t) = (\pi_1(x_t), \dots, \pi_{m(x_t)}(x_t)),$$

with $m(x_t)$ the number of actions available at x_t. The probability of a trajectory factorises into a model part and a policy part at every step:

$$\text{Prob}[x_k = i_k, \dots, x_N = i_N,\ u_k = j_k, \dots, u_{N-1} = j_{N-1}] = \prod_{\ell=k}^{N-1} \underbrace{P(x_{\ell+1} = i_{\ell+1} \mid x_\ell = i_\ell, u_\ell)}_{\text{depends on the model}} \underbrace{P(u_\ell = j_\ell \mid x_\ell = i_\ell)}_{\text{depends on the policy}}.$$

The two policies differ only in the second factor. Forming the ratio, the model factors are identical in numerator and denominator – the same trajectory, the same transitions – and cancel term by term:

$$\rho = \frac{\prod_{\ell=k}^{N-1} P(x_{\ell+1} \mid x_\ell, u_\ell)\, \pi_{j_\ell}(i_\ell)}{\prod_{\ell=k}^{N-1} P(x_{\ell+1} \mid x_\ell, u_\ell)\, \mu_{j_\ell}(i_\ell)} = \prod_{\ell=k}^{N-1} \frac{\pi_{j_\ell}(i_\ell)}{\mu_{j_\ell}(i_\ell)}.$$

The importance ratio depends **only on the two policies**, both of which we know, and not on the model, which we do not. That is what makes off-policy learning computable at all.

Quick exercise: what the cancellation does and does not buy

The model cancels from the importance ratio. Does it follow that importance sampling works equally well however different the two policies are?

Solution

No. The cancellation makes the ratio **computable**; it says nothing about whether the resulting estimator is any **good**.

Look at what the surviving product does over a horizon. The ratio is $\prod_{\ell=k}^{N-1} \pi_{j_\ell}(i_\ell)/\mu_{j_\ell}(i_\ell)$, a product of $N-k$ terms. If the two policies differ even slightly at each step, the terms are each a little above or below one, and their product has a variance that grows geometrically in the horizon. Over a long trajectory, almost all the weight ends up on a handful of samples, and the estimator – still perfectly unbiased – becomes useless.

The extreme case is instructive. If π is deterministic and μ is its ϵ-soft version, then $\pi_{j_\ell}(i_\ell)$ is 1 when the trajectory happened to follow the target and 0 otherwise, so ρ is either $1/(1-\epsilon)^{N-k}$ or exactly zero. Every trajectory that ever deviated contributes nothing at all, and the fraction that survives is $(1-\epsilon)^{N-k}$ – which for $\epsilon = 0.1$ and a fifty-step horizon is about half a percent of the data. Problem 11.8 works this case through and shows that something simpler and better is available for it.

So the cancellation is what makes the method *possible*, and the length of the product is what limits how far apart the two policies can usefully be. It is the main reason practical off-policy methods use per-step corrections, truncated or clipped ratios, or avoid importance sampling altogether – as Q-learning does.

Ordinary and weighted importance sampling

The bottleneck just identified – rare events, a small denominator, a large influence on the estimate – is mitigated by changing the normaliser.

Ordinary importance sampling divides by the number of samples:

$$\hat{J}(i) = \frac{1}{\bar{n}(i)} \sum_{r=1}^{\bar{n}(i)} \rho_r R_r(i).$$

It is **unbiased**, and it has **large variance** because of the influence of rare events.

Weighted importance sampling divides by the sum of the weights instead:

$$\hat{J}(i) = \frac{1}{\sum_{r=1}^{\bar{n}(i)} \rho_r} \sum_{r=1}^{\bar{n}(i)} \rho_r R_r(i).$$

It is **biased** – a ratio of two random quantities is not the ratio of their means – but asymptotically consistent, and it has much **smaller variance**. In applications it is the one to prefer.

The intuition is worth stating because it makes the choice obvious. In expectation the weights sum to $\bar{n}(i)$, since each has mean one. In any finite sample they do not, and the discrepancy is pure noise: it reflects how many high-ratio samples happened to arrive, not anything about the quantity being estimated. Ordinary importance sampling divides by the number that *should* have come out; weighted importance sampling divides by the one that *did*, cancelling that noise entirely. The price is that the estimator is no longer linear in the samples, hence no longer unbiased.

Problems 11.7 and 11.11 both make this visible with small numbers. In 11.7 the weights sum to 26.1 against $N = 20$, and renormalising moves the estimate from 3.8775 to 2.9713 against a true value of 2.96. In 11.11

the same arithmetic rescues an estimate of average height that ordinary importance sampling puts *below* the female average of a population that is 80% male.

Example: blackjack

Blackjack, also from OpenAI Gymnasium, is used here to compare on-policy with off-policy evaluation, and it is the right example because it is small enough to solve **exactly**, so every estimate can be checked against the truth.

The game starts with the dealer holding one card face up and one face down, and the player holding two cards face up. Face cards are worth 10; an ace counts as 11 (a *usable ace*) or as 1; numerical cards count their number. The player holds the sum of their cards and may request more (*hit*) until they decide to stop (*stick*) or exceed 21 (*bust*, an immediate loss). After the player sticks, the dealer reveals the face-down card and draws until reaching 17 or more. If the dealer busts, the player wins; if neither busts, the outcome is decided by whose sum is closer to 21.

The optimal policy is computed by dynamic programming, an ϵ-soft version of it is used as the behaviour policy, and the target policy is the optimal policy itself. Fig. 13 shows the result.

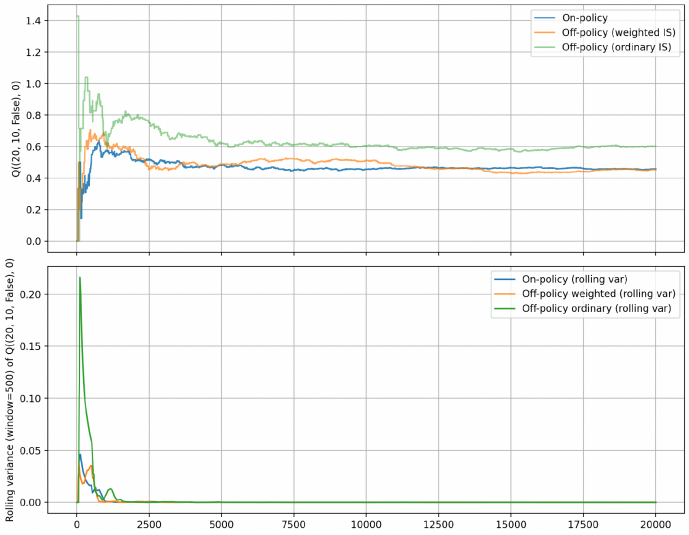


Fig. 13: On-policy and off-policy evaluation of one state's value in blackjack, with the exact value known. The off-policy variance is substantially higher than the on-policy variance, as expected from the product of importance ratios; the weighted version performs quite well.

`A_11_BlackjackOnOffPolicy.ipynb` reproduces this and adds the exact answer alongside. It estimates one Q-factor, $Q((20, 10, \text{False}), \text{stick})$, three ways – on-policy, and off-policy with ordinary and with weighted importance sampling – with $\epsilon = 0.6$. The exact value, by value iteration, is $+0.4350$, and all three estimators land on it; the difference between the two off-policy estimators is variance, not location, which is the point. The notebook also documents a one-index bug in the original script, worth knowing about because the symptom is exactly what an unbiased estimator should never show: the ordinary-IS curve settling at 0.6076 while the other two settle at 0.43.

Summary

Policy iteration is a second route to an optimal policy, quite unlike dynamic programming. It alternates **evaluating** a policy – which, for a fixed policy, is a linear system rather than a minimisation, because the min is exactly what the policy removes – with **improving** it greedily. It terminates in finitely many steps,

since there are finitely many policies and each round produces a strictly better one until it produces the same one, and when it stops it holds a certificate of optimality rather than a tolerance.

Policy improvement is exactly a rollout step with the current policy as the base policy, which is why Unit 8's material reappears here, and the rollout guarantee is what makes the iteration monotone.

Written with Q-factors, the improvement step needs no model at all – it is a minimisation over a stored table. That is what makes the whole scheme usable for reinforcement learning, and it is the same observation that made Q-learning possible in Unit 10. The model dependence is not removed but relocated, from a step that cannot be sampled to one that can.

Replacing the exact evaluation by simulation gives **simulation-based policy iteration**. The two standard estimators are **Monte-Carlo**, which is unbiased but needs whole trajectories, and **temporal difference**, which is biased but local, immediate, and reuses the estimates already built up. They are the two ends of one family rather than rivals: the Monte-Carlo error is the discounted sum of the TD errors along the trajectory, and truncating that sum interpolates between them.

Relaxing either step gives **generalized policy iteration**, the pattern behind most practical algorithms, at the cost of the finite-termination guarantee. **SARSA** is the on-policy TD control algorithm of that family; it differs from Q-learning in exactly one symbol – the bootstrap is the action actually taken next rather than the minimising one – and that symbol is the difference between evaluating the policy being followed and evaluating the greedy one.

Exploration is the recurring difficulty, and it is worse with Q-factors, since a deterministic policy visits only one action per state and no amount of process noise will visit the others. ϵ-soft policies and exploring starts are the two standard remedies, neither fully satisfactory; the first changes the policy being evaluated and the second needs a simulator that can be started anywhere.

Off-policy learning – learning about one policy while following another – is handled by importance sampling. The importance ratio is a product of per-step policy ratios in which the model cancels, which is what makes it computable; the length of that product is what limits how far apart the two policies can usefully be. Weighting the estimate by the realised weights rather than the sample count trades a little bias for a large reduction in variance, and is what one uses in practice.

> **i** Appendix: trajectory reuse and the bias-variance trade-off
>
> Monte-Carlo policy evaluation was stated above in terms of simulations of length N, with the choice of N deferred. This appendix collects what that choice involves, and one implementation device that softens it.
>
> **Trajectory reuse.** The straightforward scheme is to start at many initial states and, for each, run several simulations of large length N with $u_k^s = \mu^k(x_k^s)$, then average the returns. Fig. 14 shows the arrangement: one block of trajectories per initial state, one estimate per block.

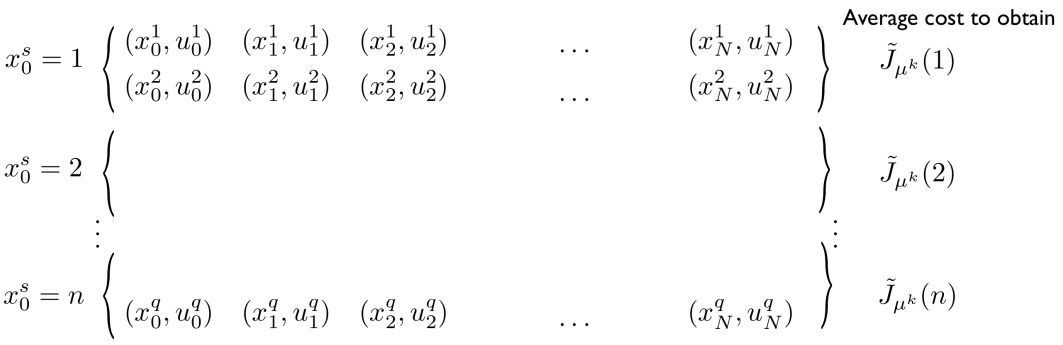


Fig. 14: The straightforward Monte-Carlo arrangement. For each initial state $x_0^s = 1, \dots, n$, several trajectories of length N are simulated under the policy μ^k, and the average of their costs gives $\tilde{J}_{\mu^k}$ at that state. With Q-factors the same picture applies with the blocks indexed by initial state-action pairs (x_0^s, u_0^s) instead.

If N is large this is wasteful, because a single long trajectory passes through many states and each passage is itself a sample of the cost-to-go from that state. Fig. 15 shows the idea: a trajectory started at state 1 that reaches state 2 at its second step supplies, from that point on, a perfectly good return for state 2 as well.

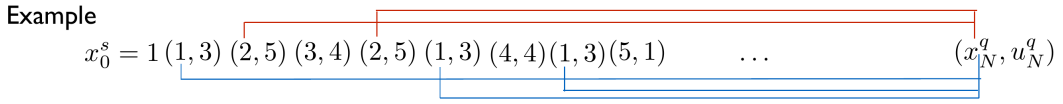


Fig. 15: Reusing one simulation for several states. The pairs are (state, action). The red bracket marks the suffix of the trajectory that contributes a return for $\tilde{J}_{\mu^k}(2)$ – everything from the first visit to state 2 onwards – and the blue brackets mark the suffixes contributing to the other states visited early on.

One subtlety is worth flagging: the returns harvested this way from a single trajectory are **not independent**, since they share their tails. Averaging them is still unbiased, but the variance does not fall as fast as the count of samples suggests. The device buys data cheaply, not freely.

How should N be chosen? The trade-off has four parts.

- A small N gives inaccurate results – **bias** – since the estimates leave out $\alpha^N g(x_N, u_N)$, $\alpha^{N+1} g(x_{N+1}, u_{N+1}), \dots$, and for a cost that is consistently signed the omission is systematic rather than averaging out.
- A large N gives accurate results but costs more computation.
- An estimate built from a small N depends on fewer random variables, so it has **smaller variance**.
- Small N has two further practical advantages: more simulations can be run for the same budget, and each can be started from an initial state of our choosing. Long trajectories may wander into a small set of states and stay there, which is poor exploration.

So the choice is not simply "as large as affordable". A short horizon is biased but cheap, low-variance and good for exploration; a long one is nearly unbiased but expensive, noisy and narrow. The discount factor sets the scale: contributions beyond N are damped by α^N, so N of the order of a few times $1/(1-\alpha)$ captures most of the mass. Problem 11.5 shows the bias concretely – a Monte-Carlo pass over an eleven-step trajectory with $\alpha = 0.9$ captures only about 69% of the discount mass, and its estimates drift downwards along the trajectory for exactly that reason.